

Lista 1

SZTUCZNA INTELIGENCJA

KAMIL TATROCKI (280506)

tło teoretyczne zastosowanych algorytmów

Algorytm Dijkstry

Pozwala wyznaczyć najszybszą trasę łączącą dwa punkty w sieci posiadającej przypisane wagi krawędzi. Cały proces opiera się na wykorzystaniu kolejki priorytetowej (kopiec), w której poszczególne węzły są porządkowane pod kątem ich czasu od punktu wyjścia. Pracę zaczynamy od umieszczenia w tej kolejce wierzchołka. Dopóki w zestawieniu pozostają jakiekolwiek elementy, system systematycznie pobiera punkt o najniższej aktualnej wadze i analizuje wszystkich jego bezpośrednich sąsiadów (połączenia z tego peronu plus z innych peronów w obrębie tej samej stacji). W każdym takim kroku sprawdza się, czy przejście przez obecnie rozpatrywany węzeł oferuje szybszą drogę do sąsiada niż ta ustalona wcześniej. Jeśli nowa trasa okazuje się korzystniejsza, wartość dystansu dla danego punktu zostaje nadpisana.

Algorytm A*

Metoda ta stanowi rozwinięcie klasycznego podejścia do szukania najoptymalniejszej drogi poprzez wprowadzenie elementu inteligencji w postaci heurystyki. Podobnie jak w rozwiązaniach bazujących na kolejce priorytetowej, system analizuje możliwe ścieżki, ale przy wyborze kolejnego węzła nie kieruje się wyłącznie dotychczas kosztem tymczasowym. Do faktycznego kosztu dotarcia do danego punktu dodawana jest funkcja heurystyczna, jaka pozostała jeszcze do celu. Dzięki temu algorytm priorytetowo traktuje te kierunki, które w teorii przybliżają go do punktu końcowego, co pozwala na znaczne ograniczenie liczby sprawdzanych połączeń. W praktyce mechanizm ten „celuje” w stronę przeznaczenia, zamiast przeszukiwać sieć we wszystkich kierunkach równomiernie.

Funkcja celu = funkcja dotychczasowy koszt + funkcja heurystyczna

Użyte heurystyki

Czas przejazdu - aproksymacji euklidesowej na rzutowanych współrzędnych

Liczba przesiadek

Podstawowa

{

0.0 - jeśli dany peron ma bezpośrednią linię do celu.

1.0 - pozostałe

}

Zaawansowana

{

0.0 - jeśli dany peron ma bezpośrednią linię do celu.

1.0 - jeśli jedna z naszych linii na tym peronie krzyżuje się gdziekolwiek na wspólnej trasie z jedną z linii dot. stacji docelowej (czyli gwarantowana jest trasa z jedną przesiadką).

2.0 - pozostałe

}

Algorytm TabuSearch

Jest to technika optymalizacyjna stosowana głównie w problemach, gdzie znalezienie idealnego rozwiązania jest zbyt kosztowne obliczeniowo. Proces polega na iteracyjnym przeszukiwaniu sąsiedztwa aktualnego rozwiązania w poszukiwaniu poprawy wyniku. Kluczowym elementem jest tutaj wykorzystanie tzw. **listy tabu**, czyli krótkotrwałej pamięci przechowującej ostatnio wykonane ruchy lub odwiedzone stany. Blokada tych elementów zapobiega powrotowi do tych samych punktów i pozwala algorytmowi „wydostać się” z optimum lokalnych, wymuszając eksplorację nowych obszarów przestrzeni poszukiwań. Jeśli algorytm utknie w miejscu, które wydaje się najlepsze w danej okolicy, ale wciąż nie jest satysfakcjonujące, lista tabu zmusza go do wykonania ruchu pogarszającego wynik, co docelowo ma doprowadzić do znalezienia jeszcze lepszej ścieżki w innym regionie.

przykład użycia metody

Dijkstra – kryterium czasu

```
PS C:\github\AI> uv run tasks/task_1_dijkstra.py
Szukanie trasy z 'Sobótka' do 'Smolec' (Tylko optymalizacja czasu przejazdu, Start: 2026-03-15 9:00)
-----
Znaleziono trasę! Najszybszy czas przyjazdu na miejsce: 11:20:00
Trasa:
[09:45:00 - 09:49:00] Sobótka -> Rogów Sobócki [Linia D4]
[09:50:00 - 09:56:00] Rogów Sobócki -> Pustków Żurawski [Linia D4]
[09:56:00 - 10:00:00] Pustków Żurawski -> Wierzbice [Linia D4]
[10:00:00 - 10:03:00] Wierzbice -> Kobierzycze [Linia D4]
[10:09:00 - 10:13:00] Kobierzycze -> Domasław [Linia D4]
[10:14:00 - 10:17:00] Domasław -> Bielany Wrocławskie [Linia D4]
[10:17:00 - 10:20:00] Bielany Wrocławskie -> Wrocław Partynice [Linia D4]
[10:21:00 - 10:24:00] Wrocław Partynice -> Wrocław Wojszyce [Linia D4]
[10:24:00 - 10:32:00] Wrocław Wojszyce -> Wrocław Główny [Linia D4]
--> [PRZESIADKA] Zmiana linii na stacji Wrocław Główny z D4 na D60 <--
[11:07:00 - 11:12:00] Wrocław Główny -> Wrocław Grabiszyn [Linia D60]
[11:13:00 - 11:14:00] Wrocław Grabiszyn -> Wrocław Zachodni [Linia D60]
[11:15:00 - 11:17:00] Wrocław Zachodni -> Mokronos Górny [Linia D60]
[11:18:00 - 11:20:00] Mokronos Górny -> Smolec [Linia D60]
PS C:\github\AI> & c:\github\AI\.venv\Scripts\Activate.ps1
(ai) PS C:\github\AI>
```

A*– kryterium czasu

```
● Rozpoczęcie szukania A*: Lubawka -> Zduny (Kryterium: t, Start: 2026-03-08 8:00)
Znaleziono trasę! Najszybszy czas przyjazdu na miejsce: 12:57:00
Trasa:
[09:31:00 - 09:36:00] Lubawka -> Błażkowa [Linia D66]
[09:37:00 - 09:43:00] Błażkowa -> Kamienna Góra [Linia D66]
[09:43:00 - 09:51:00] Kamienna Góra -> Sędziszów [Linia D66]
[WALK + PRZESIADKA] 1474758.0 -> 1474821 (przejście wewnątrz stacji)
[09:56:00 - 10:01:00] Sędziszów -> Witków Śląski [Linia D60]
[10:02:00 - 10:06:00] Witków Śląski -> Boguszków-Gorce Zachód [Linia D60]
[10:06:00 - 10:09:00] Boguszków-Gorce Zachód -> Boguszków-Gorce [Linia D60]
[10:09:00 - 10:11:00] Boguszków-Gorce -> Boguszków-Gorce Wschód [Linia D60]
[10:11:00 - 10:17:00] Boguszków-Gorce Wschód -> Wałbrzych Główny [Linia D60]
[10:17:00 - 10:22:00] Wałbrzych Główny -> Wałbrzych Fabryczny [Linia D60]
[10:24:00 - 10:27:00] Wałbrzych Fabryczny -> Wałbrzych Centrum [Linia D60]
[10:28:00 - 10:30:00] Wałbrzych Centrum -> Wałbrzych Miasto [Linia D60]
[10:31:00 - 10:35:00] Wałbrzych Miasto -> Wałbrzych Szczawienko [Linia D60]
[10:36:00 - 10:46:00] Wałbrzych Szczawienko -> Świebodzice [Linia D60]
[10:46:00 - 10:53:00] Świebodzice -> Jaworzyna Śląska [Linia D60]
[10:54:00 - 10:58:00] Jaworzyna Śląska -> Żarów [Linia D60]
[10:59:00 - 11:03:00] Żarów -> Imbramowice [Linia D60]
[11:04:00 - 11:08:00] Imbramowice -> Mietków [Linia D60]
[11:09:00 - 11:15:00] Mietków -> Kąty Wrocławskie [Linia D60]
[11:16:00 - 11:20:00] Kąty Wrocławskie -> Sadowice Wrocławskie [Linia D60]
[11:20:00 - 11:24:00] Sadowice Wrocławskie -> Smolec [Linia D60]
[11:24:00 - 11:26:00] Smolec -> Mokronos Górny [Linia D60]
[11:27:00 - 11:29:00] Mokronos Górny -> Wrocław Zachodni [Linia D60]
[11:30:00 - 11:32:00] Wrocław Zachodni -> Wrocław Grabiszyn [Linia D60]
[11:33:00 - 11:39:00] Wrocław Grabiszyn -> Wrocław Główny [Linia D60]
[WALK + PRZESIADKA] 1474679.0 -> 1474861 (przejście wewnątrz stacji)
[11:41:00 - 11:45:00] Wrocław Główny -> Wrocław Mikołajów [Linia D7/D83]
[11:46:00 - 11:50:00] Wrocław Mikołajów -> Wrocław Nadodrze [Linia D7/D83]
[11:51:00 - 11:55:00] Wrocław Nadodrze -> Wrocław Sołtysowice [Linia D7/D83]
[11:56:00 - 11:59:00] Wrocław Sołtysowice -> Wrocław Psie Pole [Linia D7/D83]
[11:59:00 - 12:04:00] Wrocław Psie Pole -> Długołęka [Linia D7/D83]
[12:04:00 - 12:14:00] Długołęka -> Oleśnica Rataje [Linia D7/D83]
[12:14:00 - 12:22:00] Oleśnica Rataje -> Dobroszyce [Linia D7/D83]
[12:23:00 - 12:28:00] Dobroszyce -> Grabowno Wielkie [Linia D7/D83]
[12:29:00 - 12:34:00] Grabowno Wielkie -> Bukowice Trzebnickie [Linia D7/D83]
[12:34:00 - 12:40:00] Bukowice Trzebnickie -> Krośnice [Linia D7/D83]
[12:40:00 - 12:46:00] Krośnice -> Milicz [Linia D7/D83]
[12:47:00 - 12:55:00] Milicz -> Cieszków [Linia D7/D83]
[12:55:00 - 12:57:00] Cieszków -> Zduny [Linia D7/D83]

Kryterium (czas podróży w sek.): 17820
Czas obliczeń: 0.2587 s
○ (ai) PS C:\github\AI>
```

A*, kryterium przesiadki, podstawowa heurystyka

```
Czas obliczeń: 0.2866 s
● (ai) PS C:\github\AI> uv run tasks/task_1_a_start.py
Rozpoczęcie szukania A*: Forst (Lausitz) -> Jerzmanki (Kryterium: p, Start: 2026-03-08 8:00)
Znaleziono trasę! Najszybszy czas przyjazdu na miejsce: 13:15:00
Trasa:
[08:40:00 - 08:44:00] Forst (Lausitz) -> Zasieki [Linia D14]
[08:44:00 - 08:54:00] Zasieki -> Tuplice [Linia D14]
[08:55:00 - 08:59:00] Tuplice -> Tuplice Dębinka [Linia D14]
[08:59:00 - 09:05:00] Tuplice Dębinka -> Lipinki Łużyckie [Linia D14]
[09:06:00 - 09:09:00] Lipinki Łużyckie -> Sieniawa Żarska [Linia D14]
[09:10:00 - 09:14:00] Sieniawa Żarska -> Żary [Linia D14]
[09:24:00 - 09:35:00] Żary -> Żagań [Linia D14]
[09:36:00 - 09:44:00] Żagań -> Małomice [Linia D14]
[09:45:00 - 09:56:00] Małomice -> Leszno Górne [Linia D14]
[10:06:00 - 10:15:00] Leszno Górne -> Wierzbowa Śląska [Linia D14]
[10:16:00 - 10:20:00] Wierzbowa Śląska -> Modła [Linia D14]
[10:20:00 - 10:26:00] Modła -> Rokitki [Linia D14]
[10:27:00 - 10:37:00] Rokitki -> Miłkowice [Linia D14]
--> [PRZESIADKA] Zmiana linii na stacji Miłkowice z D14 na D1 <--
[10:50:00 - 10:55:00] Miłkowice -> Chojnów [Linia D1]
[10:56:00 - 11:00:00] Chojnów -> Osetnica [Linia D1]
[11:00:00 - 11:04:00] Osetnica -> Okmiany [Linia D1]
[11:04:00 - 11:09:00] Okmiany -> Tomaszów Bolesławiecki [Linia D1]
[11:10:00 - 11:15:00] Tomaszów Bolesławiecki -> Bolesławiec [Linia D1]
[11:16:00 - 11:23:00] Bolesławiec -> Zebrzydowa [Linia D1]
[11:24:00 - 11:28:00] Zebrzydowa -> Zagajnik [Linia D1]
[11:28:00 - 11:33:00] Zagajnik -> Węgliniec [Linia D1]
[11:42:00 - 11:50:00] Węgliniec -> Gierałtów Wykroty [Linia D1]
[11:50:00 - 11:55:00] Gierałtów Wykroty -> Gierałtów [Linia D1]
[11:55:00 - 12:08:00] Gierałtów -> Lubań Śląski [Linia D1]
[WALK + PRZESIADKA] 1474979.0 -> 1474940 (przejście wewnątrz stacji)
[12:53:00 - 12:59:00] Lubań Śląski -> Zaręba [Linia D62]
[13:00:00 - 13:04:00] Zaręba -> Batowice Lubańskie [Linia D62]
[13:05:00 - 13:11:00] Batowice Lubańskie -> Studniska [Linia D62]
[13:11:00 - 13:15:00] Studniska -> Jerzmanki [Linia D62]

Kryterium (przesiadki): 2
Czas obliczeń: 1.4457 s
○ (ai) PS C:\github\AI> █
```

A*, kryterium przesiadki, ulepszona heurystyka

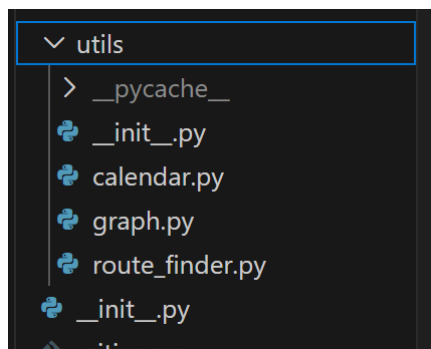
```
● (ai) PS C:\github\AI> uv run tasks/task_1_a_start.py
Rozpoczęcie szukania A*: Forst (Lausitz) -> Jerzmanki (Kryterium: p, Start: 2026-03-08 8:00)
Znaleziono trasę! Najszybszy czas przyjazdu na miejsce: 13:15:00
Trasa:
[08:40:00 - 08:44:00] Forst (Lausitz) -> Zasieki [Linia D14]
[08:44:00 - 08:54:00] Zasieki -> Tuplice [Linia D14]
[08:55:00 - 08:59:00] Tuplice -> Tuplice Dębinka [Linia D14]
[08:59:00 - 09:05:00] Tuplice Dębinka -> Lipinki Łużyckie [Linia D14]
[09:06:00 - 09:09:00] Lipinki Łużyckie -> Sieniawa Żarska [Linia D14]
[09:10:00 - 09:14:00] Sieniawa Żarska -> Żary [Linia D14]
[09:24:00 - 09:35:00] Żary -> Żagań [Linia D14]
[09:36:00 - 09:44:00] Żagań -> Małomice [Linia D14]
[09:45:00 - 09:56:00] Małomice -> Leszno Górne [Linia D14]
[10:06:00 - 10:15:00] Leszno Górne -> Wierzbowa Śląska [Linia D14]
[10:16:00 - 10:20:00] Wierzbowa Śląska -> Modła [Linia D14]
[10:20:00 - 10:26:00] Modła -> Rokitki [Linia D14]
[10:27:00 - 10:37:00] Rokitki -> Miłkowice [Linia D14]
--> [PRZESIADKA] Zmiana linii na stacji Miłkowice z D14 na D1 <--
[10:50:00 - 10:55:00] Miłkowice -> Chojnów [Linia D1]
[10:56:00 - 11:00:00] Chojnów -> Osetnica [Linia D1]
[11:00:00 - 11:04:00] Osetnica -> Okmiany [Linia D1]
[11:04:00 - 11:09:00] Okmiany -> Tomaszów Bolesławiecki [Linia D1]
[11:10:00 - 11:15:00] Tomaszów Bolesławiecki -> Bolesławiec [Linia D1]
[11:16:00 - 11:23:00] Bolesławiec -> Zembrzydowa [Linia D1]
[11:24:00 - 11:28:00] Zembrzydowa -> Zagajnik [Linia D1]
[11:28:00 - 11:33:00] Zagajnik -> Węgliniec [Linia D1]
[11:42:00 - 11:50:00] Węgliniec -> Gierałtów Wykroty [Linia D1]
[11:50:00 - 11:55:00] Gierałtów Wykroty -> Gierałtów [Linia D1]
[11:55:00 - 12:08:00] Gierałtów -> Luban Śląski [Linia D1]
[WALK + PRZESIADKA] 1474979.0 -> 1474940 (przejście wewnątrz stacji)
[12:53:00 - 12:59:00] Luban Śląski -> Zaręba [Linia D62]
[13:00:00 - 13:04:00] Zaręba -> Batowice Lubaniskie [Linia D62]
[13:05:00 - 13:11:00] Batowice Lubaniskie -> Studniska [Linia D62]
[13:11:00 - 13:15:00] Studniska -> Jerzmanki [Linia D62]

Kryterium (przesiadki): 2
Czas obliczeń: 0.7584 s
● (ai) PS C:\github\AI> █
```

opis dodanych modyfikacji do algorytmów bazowych,

Starłem się zaimplementować algorytmy zgodnie z instrukcją do listy. Modyfikacje które dodałem wyniki wprost z charakterystyki danych. Dodałem sprawdzenie czy dana trasa jest dostępna tego dnia, sprawdzenie czy nie warto poczekać na peronie przez kolejny dzień (mój kod zakłada że szuka tras max 1 dnia do przodu).

Napisałem to używając paradygmatu obiektowego. Stworzyłem klasy które były dodatkiem dla opisanych powyżej algorytmów i robiły pracę pozwalającą dopasować algorytm do struktury danych.



```

class Edge:
    def __init__(self, from_stop, to_stop, departure_time_sec, arrival_time_sec, route_short_name, route_long_name):
        self.from_stop = from_stop
        self.to_stop = to_stop
        self.departure_time_sec = departure_time_sec
        self.arrival_time_sec = arrival_time_sec
        self.travel_time = arrival_time_sec - departure_time_sec
        self.route_short_name = route_short_name
        self.route_long_name = route_long_name
        self.route_id = route_id
        self.service_id = service_id
        self.trip_id = trip_id

    def __repr__(self):
        name = self.route_short_name if pd.isna(self.route_short_name) else self.route_long_name
        return f"Edge({self.from_stop} -> {self.to_stop}, route={name}, trip={self.trip_id})"

class Graph:
    def __init__(self, data_consumer):
        self.data_consumer = data_consumer
        self.nodes = {}
        self.adjacency_list = defaultdict(list)
        self.parent_station_map = defaultdict(list)

        self.build_nodes()
        self.build_edges()

```

wykorzystane materiały i źródła (z prawidłowymi cytowaniami),

Instrukcja zadań

<https://www.youtube.com/watch?v=GazC3A4OQTE>

<https://www.youtube.com/watch?v=ySN5Wnu88nE>

<https://www.youtube.com/watch?v=tIDhFPhrCbU>

krótki opis użytych bibliotek

Pandas – z niej odczytywałem dane z plików, robiłem left join i pracowałem na danych. Jej sercem są obiekty nazywane DataFrame, które najlepiej wyobrazić sobie jako bardzo rozbudowane arkusze kalkulacyjne. Pozwalają one trzymać dane w formie tabelarycznej, gdzie każdy wiersz to osobny rekord, a kolumny mają swoje nazwy i określone typy informacji.

opis problemów napotkanych podczas implementacji.

1. Zrozumienie danych i powiązania między nimi (np. w stops.txt jest 3 razy Sobótka)
2. Ustalanie jak ma się zachowywać algorytm (np. czy trasa może trwać dłużej niż 1 dzień)
3. Szukaniu bugów w implementacji trwało bardzo długo (np. nieuwzględnienie dni tygodnia).